LeetCode 2778 — Sum of Squares of Special Elements

Language: Java

Question

You are given a 1-indexed integer array **nums** of length **n**. An element **nums[i]** is special if **i** divides **n**, meaning **n % i == 0**. Return the sum of the squares of all special elements.

Example 1

Input: nums = [1, 2, 3, 4]
Output: 21
Explanation: The special indices are 1, 2, and 4. Their squares are 1 × 1, 2 × 2, and 4 × 4. The sum is 1 + 4 + 16 = 21.

Example 2

Input: nums = [2, 7, 1, 19, 18, 3]
Output: 63
Explanation: The special indices are 1, 2, 3, and 6. Their squares add up to 2 × 2 + 7 × 7 + 1 × 1 + 3 × 3 = 63.

Java Solution

```
class Solution {
    public int sumOfSquares(int[] nums) {
        int n = nums.length;
        int sum = 0;

        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                sum += nums[i - 1] * nums[i - 1];
            }
        }

        return sum;
    }
}
```

Simple Logic to Remember

1. Find the array length: **n = nums.length**.
2. Check every index **i** from 1 to n.
3. If **n % i == 0**, that index is special.
4. Add the square of **nums[i - 1]** to sum.
5. Return sum.

Important Note

The problem uses 1-based indexing, but Java arrays use 0-based indexing. Therefore, use **nums[i - 1]** to access the element at index i in the question.

Complexity

Time: O(n)
Extra space: O(1)